PHYS30762 Object Oriented Programming in C++

Analogue Circuits Report

Kuba Kane
11019699

Department of Physics and Astronomy
The University of Manchester

May 2026

Abstract

Abstract text goes here

Contents

1 Introduction **2**

2 Code design and implementation **2**

3 Results **4**

4 Discussion and conclusions **4**

References **5**

1. Introduction

This program allows users to build analogue circuits using resistors, capacitors, and inductors (which are all classes derived from an abstract base `Component` class). A `Circuit` class can be set to be series or parallel, and any number of components can be added to it, which is then stores as a vector of components (specifically a vector of unique pointers to components). Since the `Circuit` class is inherited from the `Component` class itself, it is possible to add series circuits to parallel circuits, or vice versa, and therefore construct circuits of any topology.

In the main file of this program, a *library* of components is created, which stores different types of resistors, capacitors and inductors. These components are validated through the use of validation files, such that if the user tries to initialise a component with properties which do not match those specified in the validation file, an error message will be returned. The validation files specify a component name, like *Wirewound* for resistor, as well as a tolerance to dissipative effects, and a maximum frequency to which they can withstand [5]. Each validation file then contains maximum and minimum possible values for the resistance, capacitance and inductance respectively.

The `Component` base class contains an operating frequency `frequency` and a unique ID `component_id`, as well as a function to calculate the impedance of the component (using the complex library in C++), and its magnitude and phase difference (`get_impedance()`, `get_impedance_magnitude`, `get_phase_difference`). It also has a `base_info()` function which outputs these properties, and which is overriden in the derived classes to print the resistance, capacitance and inductance of the corresponding classes.

This program therefore gives the user an easy way of constructing circuits with nested branches and varying topologies, as well as ensuring at the same time that the components are physically meaningful, much like an undergraduate circuit labaratory where only certain types of components are available and can be used to build circuits.

2. Code design and implementation

We continue with this idea of the program being a simulation of a circuits lab so that the logic behind it can be more clearly explained. Consider the user of the program to be a student in an undergraduate circuits lab. This student wants to build a circuit using components from the lab's demonstrator, who has this library of components available. This library must contain components which are manufacturable, and we must have some way of enforcing this. The validation files can be seen as catalogues of manafacturable components from which the lab demonstrator must choose from to add to their library.

The base `Component` class contains two functions which are used by the derived classes to perform the validation checks on the components. The first is `load_validation_file` which takes in the name of the file, as well as the record to which to save it to. This record is something each derived class has, and is a vector of the `ValidationRecord` struct, whose data members are equivalent to each field in a line of the validation files (component name, maximum value, minimum value, tolerance, maximum frequency). `load_validation_file`also performs checks on the file for any blank lines or extra characters. The second function which `Component` has is `validate_component`. This function takes values from a user who wishes to intialise a new component or circuit, and checks that they match the record for the corresponding class. `validate_component` is

used within all constructors and setters of the derived classes, apart from the `Circuit` class which does not have any validation checks. That being said, the `Circuit` class does use `validate_component` if a component is added to a circuit, or its frequency if set via a setter, to check that the frequency being set does not violate the maximum frequency of a component inside a circuit. This is because when a component is added to a circuit, its frequency is set to match the circuit's frequency. In the analogy, it can be thought that the lab demonstrator is keeping the components at some set frequency in their library, which is changed when they are added to a circuit.

Within the validation function, it was useful to use a lambda function, as shown in listing 1,

```
1  //Lambda function to check for extra characters at end of
       each field
2          auto parse_double = [&line, &component_type_str](const
       string& field, const string& field_name, double& output)
3          {
4            std::stringstream field_ss(field); //Create new
       steam to capture just one field (not whole line/record)
5            string extra; //empty string to catch anything extra
       after double
6
7            //Return true if field_ss cannot extract double from
       output (the field), or if it can extract more characters
       after
8            //reading output
9            if(!(field_ss >> output) || (field_ss>>extra))
10           {
11             std::cerr<<component_type_str<<" validation file
       error: Extra or invalid characters in "<<
12               field_name<<" in the line {"<<line<<"}. Line
       will not contribute."<<std::endl;
13
14             return false; //parse failed
15           }
16
17           return true;
```

Listing 1: Lambda function in validate component function

Lambda function

If the student wants to use one of the components from the library of the lab demonstrator, they cannot take it directly away from the lab demonstrator, as they are the owner of that specific component. This follows from the fact that the the library which is set up in main is a vector of unique pointers.

IDs

If they let the student take that one component which they own, then no one else in the lab could use it. In that sense it is not logical for identical components to be stored twice in the library. The student must make a copy of the component from the library, at which point they can add it to their circuit. This is done via a `clone()` function which all of the `Component`-derived classes have. They could initialise a component just

by itself and then move it to the circuit, but its easier to just clone inside the circuit infrastructure.

3. Results

Text goes here

4. Discussion and conclusions

Text goes here

References

[1] All About Circuits. Electric fields and capacitance. https://www.allaboutcircuits.com/textbook/direct-current/chpt-13/electric-fields-capacitance/. Accessed: 16 May 2026.

[2] Boundless. 22.2: AC circuits. https://phys.libretexts.org/Bookshelves/University_Physics/Physics_(Boundless)/22%3A_Induction_AC_Circuits_and_Electrical_Technologies/22.2%3A_AC_Circuits, Sep 2018. Accessed: 16 May 2026.

[3] cplusplus.com contributors. Difference between `dynamic_cast` and `static_cast`. https://cplusplus.com/forum/unices/132444/. Accessed: 16 May 2026.

[4] Digi-Key Electronics. Capacitor kits. https://www.digikey.co.uk/en/products/filter/capacitor-kits/651. Accessed: 16 May 2026.

[5] EEPower. Resistor materials. https://eepower.com/resistor-guide/resistor-materials/. Accessed: 16 May 2026.

[6] EEPower. Types of capacitors. https://eepower.com/capacitor-guide/types. Accessed: 16 May 2026.

[7] Electronics For You. Circuit design calculator using C. https://www.electronicsforu.com/electronics-projects/software-projects-ideas/circuit-design-calculator-using-c. Accessed: 16 May 2026.

[8] Electronics Tutorials. Resistor in AC circuits. https://www.electronics-tutorials.ws/resistor/res_8.html. Accessed: 16 May 2026.

[9] Glenn Elert. Inductance. https://physics.info/inductance/, 2025. Accessed: 16 May 2026.

[10] Andreas Fertig. How to use a `constexpr std::string`. https://andreasfertig.com/blog/2025/01/how-to-use-a-constexpr-stdstring/, Jan 2025. Accessed: 16 May 2026.

[11] GeeksforGeeks. Complex numbers in C++ | set 1. https://www.geeksforgeeks.org/cpp/complex-numbers-c-set-1/, Jan 2017. Accessed: 16 May 2026.

[12] GeeksforGeeks. `push_back()` vs `emplace_back()` in C++ STL vectors. https://www.geeksforgeeks.org/cpp/push_back-vs-emplace_back-in-cpp-stl-vectors/, Mar 2021. Accessed: 16 May 2026.

[13] Interactive Mathematics. 9. impedance and phase angle. https://www.intmath.com/complex-numbers/9-impedance-phase-angle.php. Accessed: 16 May 2026.

[14] Iowa State University Center for Nondestructive Evaluation. Impedance. https://www.nde-ed.org/Physics/Electricity/impedance.xhtml. Accessed: 16 May 2026.

[15] Isaac Physics. Electrical impedance. https://isaacscience.org/concepts/cp_electrical_impedance. Accessed: 16 May 2026.

[16] Microsoft. Math constants. https://learn.microsoft.com/en-us/cpp/c-runtime-library/math-constants?view=msvc-170. Accessed: 16 May 2026.

[17] Microsoft. Type double. https://learn.microsoft.com/en-us/cpp/c-language/type-double?view=msvc-170. Accessed: 16 May 2026.

[18] MTI Instruments / Vitrek. Capacitor characteristics. https://vitrek.com/mti-instruments/knowledge-center/capacitor-characteristics/, Aug 2023. Accessed: 16 May 2026.

[19] Rod Nave. Electric field. http://hyperphysics.phy-astr.gsu.edu/hbase/electric/elefie.html. Accessed: 16 May 2026.

[20] Nick0412. Architecture of circuit solver. https://gist.github.com/Nick0412/0b9a0ba8848cb5225d5bd663dad8fd77. Accessed: 16 May 2026.

[21] Arthur O'Dwyer. Just how constexpr is C++20's std::string? https://quuxplusone.github.io/blog/2023/09/08/constexpr-string-firewall/, Sep 2023. Accessed: 16 May 2026.

[22] OpenStax. 15.2 simple AC circuits. https://openstax.org/books/university-physics-volume-2/pages/15-2-simple-ac-circuits. Accessed: 16 May 2026.

[23] OpenStax. 23.12: Inductance. https://phys.libretexts.org/Bookshelves/College_Physics/College_Physics_1e_(OpenStax)/23%3A_Electromagnetic_Induction_AC_Circuits_and_Electrical_Technologies/23.12%3A_Inductance, Jul 2016. Accessed: 16 May 2026.

[24] OpenStax. 15.8: Alternating-current circuits (summary). https://phys.libretexts.org/Bookshelves/University_Physics/University_Physics_(OpenStax)/University_Physics_II_-_Thermodynamics_Electricity_and_Magnetism_(OpenStax)/15%3A_Alternating-Current_Circuits/15.08%3A_Alternating-Current_Circuits_(Summary), Nov 2017. Accessed: 16 May 2026.

[25] Madhawa Polkotuwa. Understanding static and dynamic casting in C++. https://madhawapolkotuwa.medium.com/understanding-static-and-dynamic-casting-in-c-b73cf3f92ec2, Jan 2025. Accessed: 16 May 2026.

[26] RS Components. Aluminium electrolytic capacitors datasheet. https://docs.rs-online.com/49da/0900766b80fa3a92.pdf. Accessed: 16 May 2026.

[27] Scribd. Kirchhoff's law in C program. https://es.scribd.com/document/128210104/kirchoff-law-in-c-program. Accessed: 16 May 2026.

[28] Stack Overflow contributors. Answer: Why does std::vector need the move constructor to be noexcept? https://stackoverflow.com/a/20521414. Accessed: 16 May 2026.

[29] Stack Overflow contributors. C++ design for circuit solver. https://stackoverflow.com/questions/59160172/c-design-for-circuit-solver. Accessed: 16 May 2026.

[30] Wikipedia contributors. Electrical reactance. https://en.wikipedia.org/wiki/Electrical_reactance. Accessed: 16 May 2026.

[31] Wikipedia contributors. Nodal analysis. https://en.wikipedia.org/wiki/Nodal_analysis#Matrix_form_for_the_node-voltage_equation. Accessed: 16 May 2026.